# CHAPTER 19

## NETWORK SEARCH

*A discussion of methods for searching networks for particular vertices or items, a process important for web search and peer-to-peer networks, and for our understanding of the workings of social networks*

**I**N CHAPTER 4 we saw a number of examples of networks that have information stored at their vertices: the World Wide Web, citation networks, peer-to-peer networks, and so forth. These networks can store large amounts of data but those data would be virtually useless without some way of searching through them for particular items. So important is it to be able to perform fast and accurate searches that the companies that provide the most popular search services are now some of the largest in their respective industries—Google, Thomson Reuters, LexisNexis—and constitute multibillion dollar international operations. In this chapter we examine some of the network issues involved in efficient searching and some implications of search ideas for the structure and behavior of networks.

## 19.1 WEB SEARCH

We have already discussed briefly some aspects of how web search engines work in Sections 4.1, 7.4, and 7.5. In this section we discuss the issue in more detail.

Traditional, or offline, web search is a multistage process. It involves first "crawling" the Web to find web pages and recording their contents, then creating an annotated index of those contents, including lists of words and estimates of the importance of pages based on a variety of criteria. And then there is the search process itself, in which a user submits a text query to a search engine and the search engine extracts a list of pages matching that query from the index.

The process of web crawling by which web pages are discovered is interesting in itself and exploits the network structure of the Web directly. The crawler follows hyperlinks between web pages in a manner similar to the breadth-first search algorithm for finding components described in Section 10.3. The basic process is described in Section 4.1. Practical web crawlers for big search operations employ many elaborations of this process, including:

- Searching in parallel at many locations on the Web simultaneously using many different computers,
- Placing the computers at distributed locations around the world to speed access times to pages coming from different places,
- Repeatedly crawling the same web pages at intervals of a few days or weeks to check for changes in page contents or pages that appear or disappear,
- Checking on pages more often if their contents have historically changed more often,
- Checking on pages more often if they are popular with users of the search engine,
- Heuristics to spot dynamically generated pages that can lead a crawler into an infinite loop or tree of pages and waste time,
- Targeted crawling that probes more promising avenues in the network first, and
- Altered behavior depending on requests from owners of specific sites, who often allow only certain crawlers to crawl their pages, or allow crawlers to crawl only certain pages, in order to reduce the load on their servers.

At their heart, however, most web crawlers are still dumb animals, following links and recording what they see for later processing.

The processing of the raw crawler output also has interesting networkrelated elements. Early search engines simply compiled indexes of words or phrases occurring in web pages, so one could look up a word and get a list of pages containing it. Pages containing combinations of words could also be found by taking the sets of pages containing each individual word in the combination of interest and forming the intersection of those sets. Indexes can be extended by adding annotations indicating, for example, how often a word appears on a page or whether it appears in the page title or in a section heading, which might indicate a stronger connection between that word and the subject matter of the page. Such annotations allow the search engine to make choices about which are the pages most relevant to a given query. Even so, search engines based solely on indexes and textual criteria of this sort do not return very good results and have been superseded by more sophisticated technology.

Modern search engines do still use indexes in their search process, but only as a first step. A typical modern search engine will use an index to find a set of candidate pages that might be relevant to the given query and then narrow that set down using other criteria, some of which may be network-based. The initial set is usually chosen deliberately to be quite broad. It will typically

include pages on which the words of the query appear, but also pages on which they don't appear but that link to, or are linked to by, pages that do contain the query words. The net result is a set of pages that probably includes most of those that might be of interest to the user submitting the query, but also many irrelevant pages as well. The strength of the search engine, its ability to produce useful results, therefore rests primarily on the criteria it uses to narrow the search within this broad set.

The classic example of a criterion for narrowing web searches comes from the Google search engine, which makes use of the eigenvector centrality measure known as PageRank, discussed in Section 7.4. PageRank accords pages a high score if they receive hyperlinks from many other pages, but does so in a way such that the credit received for a link is higher if it comes from a page that is itself highly ranked. PageRank, however, is only one of many elements that go into the formula Google uses to rank web pages. Others include traditional measures such as frequency of occurrence of query words in the page text and position of occurrence (near the top or bottom, in titles and headings, etc.), as well as occurrence of query words in "anchor text" (the highlighted text that denotes a hyperlink in a referring page) and previous user interest in a particular page (whether people selected this page from the list of search results on other occasions when the same text query, or a similar one, was entered).

Google gives each web page in the initial set a score that is a weighted combination of these elements and others. The particular formula used is a closely guarded secret and is moreover constantly changing, partly just to try and improve results, but also to confound the efforts of web page creators, who try to increase their pages' ranking by working out what particular elements carry high weight in Google's formula and incorporating those elements into their pages.

An important point to appreciate is that some parts of the score a page receives depend on the particular search query entered by the user—frequency of occurrence of query words, for instance—but others, such as PageRank, do not. This allows Google's computers (or their counterparts in other search companies) to calculate the latter parts "offline," meaning they are calculated ahead of time and not at the time of the query itself. This has some advantages. PageRank, for instance, is computationally intensive to calculate and it saves a lot of time if you only have to calculate it once. But there are disadvantages too. PageRank measures the extent to which people link to a given web page, but people may link to a page for many reasons. Thus a page may have a high PageRank for a reason unrelated to the current search query. A page whose text makes mention of two or more different topics (and many do) may be a crucial authority on one topic but essentially irrelevant on another, and PageRank cannot distinguish between the two.

One could imagine a version of PageRank that was specific to each individual query. One could calculate a PageRank score within just the subnetwork formed by the set of pages initially selected from the index to match the query. But this would be computationally expensive and it's not what Google does. As a result it is not uncommon for a page to be ranked highly in a particular search even though a casual human observer could quickly see that it was irrelevant to the search topic. In fact, a large fraction of "bad" search results returned by search engines probably fall in this category: they are pages that are important in some context, but not in the context of the specific search conducted.

The overall process behind searches on Google and similar large search engines is thus as follows [55]. First the Web is crawled to find web pages. The text of those web pages is processed to created an annotated index, and the link structure of the hyperlinks between them is used to calculate a centrality score or scores for each page, such as PageRank in Google's case or (presumably) some similar measure for other search engines. When a user enters a query the search engine extracts a deliberately broad set of matching pages from the index, scores them according to various query-specific measures such as frequency of occurrence of the query words, then combines those scores with the pre-computed centrality measure and possibly other pre-computed quantities, to give each page in the set an overall score. Then the pages are sorted in order of their scores and the ones with the highest scores are transmitted to the user. Typically only a small number of the highest-scoring pages are transmitted—say the first ten—but with an option to see lower-scoring pages if necessary.

Despite the reservations mentioned above, this system works well in practice, far better than early web search engines based on textual content alone, and provides useful search results for

millions of computer users every day.

## 19.2 SEARCHING DISTRIBUTED DATABASES

Some information networks form distributed databases. A typical example is a peer-to-peer file-sharing network, in which individual computers in the network each store a subset of the data stored in the network as a whole. The form and function of peer-to-peer networks were described in Section 4.3.1.

The "network" in a peer-to-peer network is typically a virtual one, in which individual computers maintain contacts with a subset of others, which are not necessarily those with which they have direct physical data connections. In this respect peer-to-peer networks are somewhat similar to the World Wide Web, in which the hyperlinks between websites are virtual links chosen by a page's creator and their topology need have nothing to do with the topology of the underlying physical Internet. Indeed, the World Wide Web is itself, in a sense, a distributed database, storing information in the pages at its vertices, but web search works in a fundamentally different way from search in other distributed databases, so we treat the two separately.

Search is a fundamental problem in peer-to-peer networks and similar systems: how do we find specific items among those stored at the many vertices of the network? One way would be to copy the web search approach of Section 19.1 and construct a comprehensive index of all items at some central location and then search that index for items of interest. For a variety of reasons, however, most peer-to-peer networks don't go this route, but instead make use of distributed search techniques in which the search task is shared among the computers in the network via messages passed along network edges. Indeed the performance of such distributed searches is the primary reason for linking the vertices into a network in the first place and there are some interesting principles relating the structure of the network to the efficiency with which searches can be performed.

Suppose that we have a peer-to-peer network composed of $n$ individual computers and each computer is linked by virtual connections to a selection of the others, where "linked" in this context merely means that these others are the ones with which a computer has agreed to communicate directly in the course of performing searches. There is no reason in principle why a computer could not communicate with *all* others if it wanted to, but in practice this would demand too much effort or data bandwidth, and limiting the number of network neighbors a computer has brings the resources required within reasonable bounds.

The simplest form of distributed search, used in some of the earliest peer-to-peer networks, is a version of the breadth-first search algorithm described in Section 10.3 (where it was used for finding network components and shortest paths). Under this approach, a user gives the computer a search term, such as the name of a computer file, and the computer sends a query to each of its neighbors in the peer-to-peer network, asking if they have the file in question. If they do, they send the file to the first computer and the search is complete. If they don't, then they send a further query to each of *their* neighbors asking for the file. Any neighbor that has seen the query before, such as the computer that originated it in the first place, ignores it. All others check to see if the have the requested file and send it back to the originating computer if they do. If not, they pass the query on to their neighbors, and so on.

This simple strategy certainly works and it has some advantages. For instance, assuming that the network displays the small-world effect (Section 8.2), the number of steps we will have to take in our breadth-first search will be small even when the network is large (typically increasing only logarithmically with $n$—see Section 12.7). This means that most searches will take only a short amount of time to find the desired file.

But there are some serious disadvantages with the approach as well. First, as we have described it the search doesn't actually stop when the target file is found. There is no mechanism to inform

computers that the file has been found and that they don't need to pass the query on to anyone further. This problem can be fixed relatively easily, however, for example by requiring each computer receiving the query to check with the originating computer to see if the file has been found before they do anything else.

A more serious problem is that the messages transmitted in the process of spreading a query across the network quickly add up to a huge amount of data and can easily overwhelm the capacity of the computers involved. Assuming a worst-case scenario in which a desired file exists on only a single computer in the network, we will, on average, have to pass our query to half of all computers before we find the file. That means the number of messages sent in the course of one query is $O(n)$. Suppose that users perform queries at some constant average rate $r$, so that the overall rate of queries is $rn = O(n)$. Then the total number of messages sent per unit time is $O(n) \times O(n) = O(n^2)$ and the number of messages per computer per unit time is, on average, $O(n^2)/n = O(n)$, which goes up linearly with the size of the network. This means that, no matter how much bandwidth our computers have to send and receive data, it will in the end always become swamped if the network becomes large enough. And peer-to-peer networks can become extremely large. Some of the largest have millions of users.

Luckily this worst-case scenario is not usually realized. It is in fact rarely the case that an item of interest exists on only one computer in a network. Most items in typical peer-to-peer networks exist in many places. Indeed, assuming that some fraction of the user population likes or needs each item, it is more reasonable that any given item appears on some fixed fraction $c$ of the vertices in the network, so that the total number of copies $cn$ goes up as the size of the network increases. If this is the case, and assuming for the moment that the value of $c$ is the same for every item, then one will have to search on average only $1/c$ vertices before finding a copy of an item. This means that the total number of query messages sent over the network per unit time is $O(n/c)$ and the number per computer per unit time is $O(1/c)$, which is just a constant and does not increase with increasing network size.

A more realistic calculation allows for the fact that some items are more popular than others. Suppose that the factors $c$, which are proportional to popularity, have a distribution $p(c)$, meaning that the probability of falling in the interval $c$ to $c + dc$ is $p(c)\,dc$. Also important to note is that not all items are searched for with equal frequency. Indeed a more reasonable assumption is that they are searched for with frequency proportional to their popularity, i.e., that the probability of a search query asking for an item with popularity in the interval $c$ to $c + dc$ is $cp(c)\,dc/\langle c\rangle$, where the factor of $\langle c\rangle = \int cp(c)\,dc$ insures that the distribution is properly normalized. Then the average number of vertices we have to examine before we find the item corresponding to a typical query is

$$\int_0^1 \frac{1}{c}\,\frac{cp(c)\,dc}{\langle c\rangle} = \frac{1}{\langle c\rangle},$$

**(19.1)**

and hence the number of query messages sent or received per computer per unit time is $O(1/\langle c\rangle)$, which is again a constant as network size becomes large.

In principle, therefore, if a node can handle messages at the rate given by Eq. (19.1) then the network should go on functioning just fine as its size becomes large. In practice, however, there are still problems. The main difficulty is that vertices in the network vary enormously in their bandwidth capabilities. Most vertices have relatively slow communications with the network, i.e., low bandwidth, while a few have much better, higher-bandwidth connections. This means that even if bandwidth requirements per vertex are reduced to a constant as above, the network will still run at a speed dictated by the majority slow vertices, making queries slow and possibly overwhelming the capacity of some vertices.

To get around this problem, most modern peer-to-peer networks make use of *supernodes* (also called *superpeers*). Supernodes are high-bandwidth nodes chosen from the larger population in the network and connected to one another to form a supernode network over which searches can be performed quickly—see Fig. 19.1.

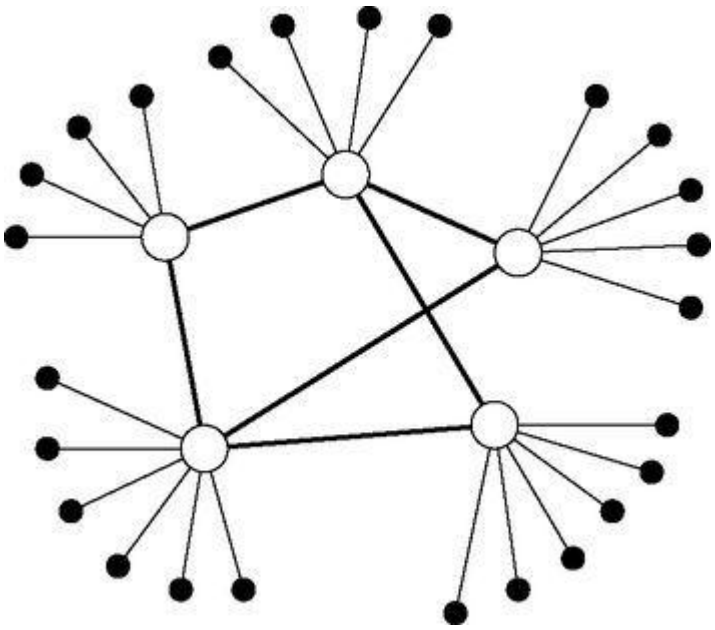


**Figure 19.1: The structure of a peer-to-peer network with supernodes.** Client nodes (filled circles) are connected to a network of supernodes (open circles) that have aboveaverage network bandwidth and hence can conduct searches quickly.

A supernode acts a little like a local exchange in a telephone network (see Section 2.2). Each normal user, or *client*, in the network attaches to a supernode (or sometimes to more than one) that acts as their link to the rest of the network. Each supernode has a number of such clients and the clients communicate to the supernode a list of the files or other data items they possess so that the supernode can respond appropriately to search queries from other supernodes. An individual client wanting to perform a search then sends their search query to the local supernode, which conducts a breadth-first search interrogation of the network of supernodes to find the desired item. Since the supernodes possess records of all the items that the clients have, the entire search can be performed on the network of supernodes alone and no client resources are used at all. And since the supernodes are deliberately selected to have fast network connections, the search runs at the speed of the quickest vertices in the network.

In practice schemes like this work quite well—well enough to be in wide use in peer-to-peer networks of millions of users. More sophisticated schemes have been devised that in theory could work better still—an example is the "Chord" system proposed by Stoica *et al.* [305]—but such systems have yet to find widespread adoption since the more traditional supernode approach appears to work well enough for practical purposes.

## 19.3 MESSAGE PASSING

A different variation of the distributed search problem is the problem of getting a message to a particular node in a network. The classic example of this problem is Stanley Milgram's "small-world" experiment, described in Section 3.6. In this experiment participants were asked to get a message to a specific target individual by passing it from acquaintance to acquaintance through the social network. Milgram famously found that messages that arrived at the destination passed through only about six people on their way, which is the origin of the popular concept of the "six degrees of separation." As discussed in Section 3.6, however, there is another perhaps more surprising implication of the experiment, first pointed out by Kleinberg [177], which is that short paths not only exist in the network but that people are remarkably good at finding them. Of course if one knows the structure of an entire network then one can find short paths directly using, for example, the breadth-first search method of Section 10.3.5. Participants in Milgram's experiment, however, did not know the whole network and probably only knew a very small part of it, and yet they were still able to get a message rapidly to the desired target.

This observation raises a number of interesting questions. How, in practice, did people find these short paths to the target? Can we come up with an algorithm that will do the job efficiently? How does the performance of that algorithm depend on the structure of the network? In the following sections we consider two different models of the message passing process that address these questions. As we will see, these models suggest that social (or other) networks must have a very particular type of structure if one wants to be able to find short paths easily without a global knowledge of the network.

### 19.3.1 KLEINBERG'S MODEL

The instructions to the participants in Milgram's experiment were that upon receiving the message (actually a small booklet or "passport" sent through the mail), they were to forward it to an acquaintance who they believed to be closer to the target than they were. The definition of "closer" was left vague, however, and one of the first things we need to do if we want to model the mechanics of the experiment is decide on a practical definition.

An illuminating attempt at modeling Milgram's experiment was made by Kleinberg [177, 178], who employed a variant of the small-world model of Section 15.1, as shown in Fig. 19.2. As in the standard small-world model, it has a ring of vertices around the edge plus a number of "shortcut" edges that connect vertex pairs at random points around the ring. In Kleinberg's model all vertices are connected to their two immediate neighbors around the ring—$c = 2$ in the notation of Chapter 15—and Kleinberg made use of the connections in the ring to define the "closeness" of vertices for the purposes of the message-passing experiment. He proposed that individuals in the network, represented by vertices, are aware of the distance around the ring to other individuals, and hence can say when one of their acquaintances is "closer" to the target vertex than they are in this sense.

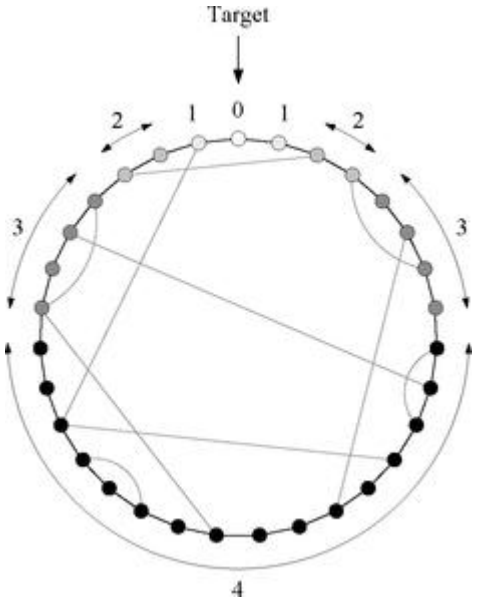



**Figure 19.2: The variant small-world model used to model message passing**.In the variant of the small-world model used here, vertices are connected around a ring and shortcuts added between them as in the normal small-world model. However, the shortcuts are now biased so that there are more of them connecting nearby vertices than distant vertices. The strength of the bias is controlled by the parameter $\alpha$. In the proof given in the text, the vertices are divided into numbered classes, class 0 consisting of just the target vertex and higher classes radiating out from the target, each successive class containing twice as many vertices as the previous one.

In his calculations Kleinberg considered a *greedy algorithm* for message passing in which each individual receiving a message passes it on to the one of their neighbors who is closest to the target in the sense above. This algorithm is guaranteed always to get the message to the target eventually.

Every individual has at least one neighbor who is closer to the target in the Kleinberg sense than they are—their neighbor around the ring in the direction towards the target. Thus on each step of the message passing process the message is guaranteed to get at least one step closer to the target around the ring and hence it must eventually get to the target. In the worst case individuals simply pass the message around the ring until it reaches its destination but generally we can expect to do better than this because of the shortcuts. The question is how much better. Kleinberg showed that it is possible for the greedy algorithm to find the target vertex in $O(\log^2 n)$ steps, but that it can do so only for particular choices of the arrangement of the shortcuts.

Kleinberg considered a one-parameter family of models that generalizes the standard small-world model by allowing for different arrangements of the shortcuts.[268] Instead of assuming that shortcuts are placed uniformly at random, we assume (not unreasonably) that people have more acquaintances among those close to them (in the sense defined above) than among those far away. By analogy with the standard small-world model let us place shortcuts around the ring equal in number to $p$ times the number of edges in the ring itself, which in this case is just $n$. Since each shortcut has two ends this means that the average number of shortcuts attached to each vertex will be $2p$ (and the actual number will be Poisson distributed with mean $2p$). Where we differ from the standard small-world model is in how these shortcuts are placed. Shortcuts are still placed at random, but they are chosen so that the probability of a particular shortcut covering a distance $r$ around the ring is $Kr^{-\alpha}$, where $\alpha$ is a non-negative constant and $K$ is a normalizing constant. That is, for each shortcut we choose first its length $r$ from this distribution, then we place the shortcut, spanning exactly $r$ vertices, at a position around the ring chosen uniformly at random. If $\alpha = 0$ then we recover the standard small-world model of Section 15.1, but more generally, for $\alpha > 0$, the model has a preference for connections between nearby vertices.

Note that the probability that a particular shortcut connects a specific pair of vertices a distance $r$ apart is equal to $Kr^{-\alpha}/n$, which is the probability $Kr^{-\alpha}$ that the shortcut has length $r$ multiplied by the probability $1/n$ that out of the $n$ possible choices it falls in the specific position around the ring that connects the two vertices in question. Given that there are $np$ shortcuts in the whole network, this means that the total probability of having a shortcut between a given pair of vertices is $np \times K^{-\alpha}/n = pKr^{-\alpha}$. (More correctly, this is the expected number of such shortcuts, but so long as the number is small, the difference is negligible.)

The normalizing constant $K$ is fixed by the condition that every shortcut must have *some* length, and that all lengths lie between 1 and $\frac{1}{2}(n-1)$, so that

$$K \sum_{r=1}^{\frac{1}{2}(n-1)} r^{-\alpha} = 1.$$

**(19.2)**

We can approximate the sum by an integral using the trapezoidal rule of Eq. (14.115) thus:

$$\begin{aligned} \sum_{r=1}^{\frac{1}{2}(n-1)} r^{-\alpha} &\simeq \int_1^{\frac{1}{2}(n-1)} r^{-\alpha}\,dr + \tfrac{1}{2} + \tfrac{1}{2}\left[\tfrac{1}{2}(n-1)\right]^{-\alpha} \\ &= \frac{\left[\frac{1}{2}(n-1)\right]^{1-\alpha} - 1}{1-\alpha} + \tfrac{1}{2} + \tfrac{1}{2}\left[\tfrac{1}{2}(n-1)\right]^{-\alpha}, \end{aligned}$$

**(19.3)**

which gives

$$K \simeq \begin{cases} (1-\alpha)(\frac{1}{2}n)^{\alpha-1} & \text{for } \alpha < 1, \\ 1/\ln \frac{1}{2}n & \text{for } \alpha = 1, \\ 2(\alpha-1)/(\alpha+1) & \text{for } \alpha > 1, \end{cases}$$

**(19.4)**

as $n$ becomes large.[270]

We can now show that, for suitable choice of $\alpha$, the greedy algorithm on this network can indeed find a given target vertex quickly. The proof is as follows. Suppose, without loss of generality, that the target vertex is at the top of the ring, as depicted in Fig. 19.2, and let us divide up the other vertices into classes according to their distance from the target. Class 0 consists of just the target itself. Class 1 consists of all vertices distance $d = 1$ from the target around the ring, of which there are two. Class 2 consists of vertices with distances in the range $2 \le d < 4$, class 3 of vertices $4 \le d < 8$, and so forth. Each class is double the size of the previous one. In general, class $k$ consists of vertices at distances $2^{k-1} \le d < 2^k$ and contains $n_k = 2^k$ vertices. (For simplicity, let us assume that the total number $n$ of vertices is a power of two, minus one, so that everything works out neatly.)

Now consider a message being passed through the network according to the greedy algorithm described above and suppose that at a particular step of the process the message is at a vertex of class $k$. How many more steps will it take before the message leaves class $k$ and passes into a lower class? The total number of vertices in lower classes is

$$\sum_{m=0}^{k-1} n_m = \sum_{m=0}^{k-1} 2^m = 2^k - 1 > 2^{k-1},$$

**(19.5)**

and from Fig. 19.2 we can see that all of these are, at most, a distance $3 \times 2^k - 2 < 2^{k+2}$ from the vertex in class $k$ that currently holds the message. Thus the probability of the vertex with the message having a shortcut to a particular one of these vertices in lower classes is at least $pK\,2^{-(k+2)\alpha}$, and the probability of having a shortcut to any of them is at least $pK\,2^{k-1-(k+2)\alpha}$.

If our vertex has no shortcut that takes the message out of class $k$ then, in the worst case, it simply passes the message to another vertex in class $k$ that is closer to the target, either via a shortcut or by passing around the ring. Using the probability above, the expected number of such moves made before we find a shortcut that takes us out of class $k$ is at most

$$\frac{1}{pK\,2^{k-1-(k+2)\alpha}} = \frac{1}{pK}\,2^{2\alpha+1}2^{(\alpha-1)k}.$$

**(19.6)**

Finally, again in the worst case, the message will pass through each of the classes before

reaching the target. There are $\log_2(n+1)$ classes in total and summing over them we find that an upper bound on the expected number of steps $\ell$ needed to reach the target is

$$\ell \le \frac{1}{pK} 2^{2\alpha+1} \sum_{k=0}^{\log_2(n+1)} 2^{(\alpha-1)k} = \frac{1}{pK} 2^{2\alpha+1} \frac{2^{(\alpha-1)[1+\log_2(n+1)]} - 1}{2^{\alpha-1} - 1}$$
$$= \frac{1}{pK} 2^{2\alpha+1} \frac{[2(n+1)]^{\alpha-1} - 1}{2^{\alpha-1} - 1}.$$

**(19.7)**

Making use of Eq. (19.4) for the constant $K$ and taking the limit of large $n$ we find that asymptotically

$$\ell \le \begin{cases} A\, n^{1-\alpha} & \text{if } \alpha < 1, \\ B \log^2 n & \text{if } \alpha = 1, \\ C\, n^{\alpha-1} & \text{if } \alpha > 1, \end{cases}$$

**(19.8)**

where $A$, $B$, and $C$ are constants depending on $\alpha$ and $p$, but not $n$, whose rather complicated values we can work out from Eqs. (19.4) and (19.7) if we want.

Since Eq. (19.8) gives an upper bound on $\ell$, this result guarantees that for the particular case $\alpha = 1$ we will be able to find the target vertex in a time that increases as $\log^2 n$ with the size of the network. This is not quite as good as $\log n$, which is the actual length of the shortest path in a typical network, but it is still a slowly growing function of $n$ and it would be fair to claim that the small-world experiment would succeed in finding short paths in a network that had $\alpha = 1$. Thus it is possible, provided the network has the correct structure, for a simple strategy like the greedy algorithm, in which vertices have knowledge only of their immediate network neighborhood, to produce results similar to those observed by Milgram in his experiment.

On the other hand, if $\alpha \neq 1$ then Eq. (19.8) increases as a power of $n$, suggesting that it would take much longer in such networks to find the target vertex. In particular, for the original small-world model of Section 15.1, which corresponds to $\alpha = 0$, Eq. (19.8) grows linearly with $n$, suggesting that the Milgram experiment could take millions of steps to find a target in a social network of millions of people. Equation (19.8) is only an upper bound on the time taken, so if one is lucky one may be able to find the target faster. For instance, if the message starts at a vertex that happens to have a shortcut directly to the target vertex then one can find the target in a single step. However, Kleinberg [178] was also able to prove that the *average* time it takes to find the target increases at least as fast as a power of $n$ except in the special case $\alpha = 1$, so in general the greedy algorithm for $\alpha \neq 1$ will not work well.[271]

These results tell us two things. First, they tell us that it is indeed possible for the small-world experiment to work as observed even if the participants don't know the details of the whole network. Second, they tell us that, at least within the context of the admittedly non-realistic model used here, the experiment only works for certain very special values of the parameters of the network. Thus the success of Milgram's experiment suggests not only that, as Milgram concluded, there are short paths in social networks, but also that they have a particular structure that makes path finding possible.

### 19.3.2 A HIERARCHICAL MODEL OF MESSAGE PASSING

While interesting, the results of the previous section are not wholly convincing because the model is clearly not a realistic one. People don't live around a circle with just a few shortcuts to others, and message passing doesn't work because people know where others live on the circle.

So can we derive similar results for a more realistic network model? To answer this question let us first ask how message passing *does* work. We can get a hint from the "reverse small-world" experiments of Killworth and Bernard [39, 174] discussed in Section 3.6. Recall that in these experiments researchers asked subjects to imagine that they were participating in Milgram's small-world experiment and then asked them what information they would want to know about the target in order to make a decision about who to pass their message on to. Killworth and Bernard found that three pieces of information were sought more often than any others, and by almost all subjects: the name, occupation, and geographic location of the target.

The target's name is an obvious requirement in the small-world experiment, since it's needed to recognize the target when you find him or her. Beyond that, however, it probably doesn't play much role in the message passing, except perhaps in cultures where names can give a clue as to the location or social status of an individual. Occupation and geographic location, on the other hand, are of great use in deciding how to forward a message, and these appear to be the primary pieces of information participants in the experiment use.

Take geographic location as an example. How would one use information on geography to route a message? Presumably, one would attempt to pass the message to someone closer geographically to the target than oneself. Suppose for instance that the target lives, as Milgram's did, in a suburb of the city of Boston, Massachusetts, in the United States. A participant in, say, England, attempting to get a message to this target, would perhaps first forward it to someone they knew in the US. That person might forward it in turn to someone they knew in the state of Massachusetts, who would forward it to someone in Boston, who would forward it to the target's specific suburb, and so forth. At each step in the process, the participants narrow down the search to a smaller and smaller geographic area until, with luck, the area is so small that someone there knows the target individual directly.

In a sense, this is what happens in Kleinberg's model. In Section 19.3.1 we divided Kleinberg's circle into zones or classes that get ever smaller as they close in on the target and showed that under suitable circumstances it takes only a small number of steps of the message-passing process to find a connection from one class to the next smaller one. Since the number of classes is logarithmic in the size of the network, this means that it also takes only a small number of steps overall to home in on the target. Kleinberg's network structure was unrealistic, but the basic idea of progressively narrowing the field is a good one and we would like to find a more realistic network model to which the same type of argument can be applied.

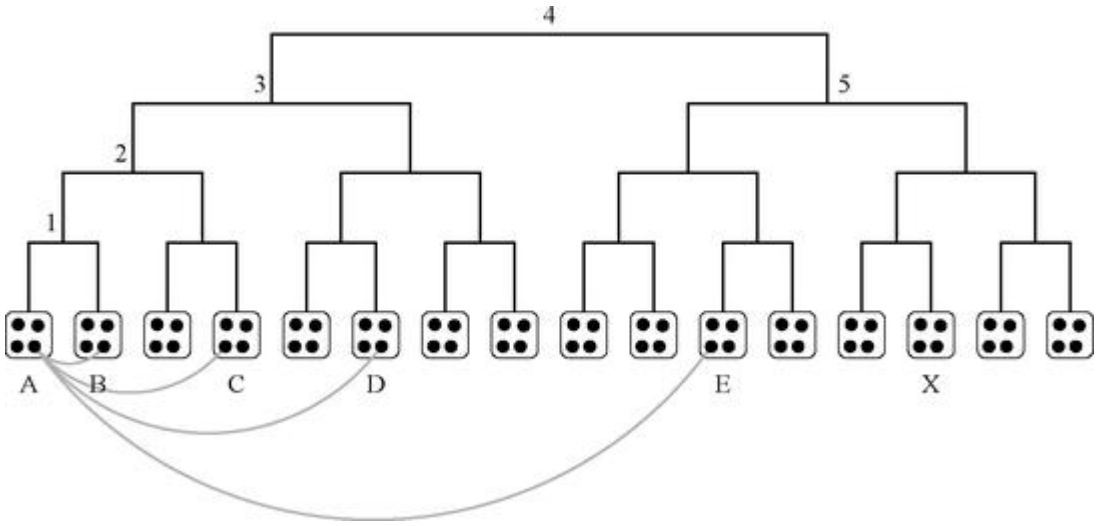



**Figure 19.3: The hierarchical model of Watts *et al.*** Small groups of individuals (boxes) are divided up in a hierarchical structure represented by a binary tree, which might, for instance, correspond to the hierarchical division of geographic space into countries, regions, towns, and so forth. The hierarchy dictates which social connections (indicated by curves) are most likely. A vertex in group A, for instance, is most likely to be connected to others close to it in the tree (B, C) and less likely to be connected to those further away (D, E).

Such a model is the hierarchical model of Watts *et al.* [322], in which the interplay of social structure and geographic or other dimensions is represented by a tree or dendrogram.[272] In the context of geography, for example, the world would be divided into countries, the countries into regions, states, or provinces, the regions into cities and towns, and so forth. The division ends when we reach units so small that it can reasonably be assumed that everyone knows everyone else—a single family, for instance.

The divisions can be represented by a tree structure like that shown in Fig. 19.3. The tree depicted is a binary tree in this case. Each branch splits in two, then in two again, and so forth. In the real world branches might easily split into more than two parts. There are more than two countries in the world after all. However, the binary tree is the simplest case to study (and the one studied by Watts *et al.*), and the analysis given here for the binary case can be generalized to other cases quite easily.

Let us also assume that the groups at the bottom of the tree all have the same size $g$. Again this is a simplification, but a useful one that does not have a major effect on the results. If the total number of individuals in the network is $n$ then the number of groups is $n/g$, and the number of levels in the tree is $\log_2(n/g)$.

The model of Watts *et al.* makes two other important assumptions. First, it assumes that people measure distance to a target individual in terms of the tree, and more specifically in terms of the lowest common ancestor in the tree that they share with the target. That is, people are able to tell when someone lives in the same country as themselves, or the same region or town, but do not have any detailed information beyond that. This is a more conservative assumption than is made by Kleinberg's model. In Kleinberg's model it is assumed that people know their exact geometric distance to the target, no matter where in the network the target falls. In the present model people have a much more coarse-grained impression of how close they are to the target.

The second assumption in the model of Watts *et al.* is that the social network itself is correlated with the hierarchical tree structure so that people who are closer together in the tree, in the sense of sharing a lower common ancestor, are also more likely to be acquainted. Thus people are more likely to know others in their own country than in other countries, more likely to know others in

their own town than in other towns, and so forth. A few sample acquaintances are represented by the curves at the bottom of the figure.

Thus there are really two networks present in this model. There is the "real" network of actual acquaintances represented by the curves, and a "shadow" network, the hierarchical tree, which is not a network of actual acquaintances but which influences the acquaintance network and of which individuals are somewhat aware, in the sense that they know how close they are to others in the tree.

An important point to note about this model is that although an individual is less likely to know others far away in the tree, there are also more such faraway individuals than there are ones close by, and the two effects cancel out to some extent so that it is quite possible for a given individual to know others who are both near and far. The people who live on your street, for instance, are close by, so you are likely to know some of them, but they are also few in number. By contrast, India may be far away for you (depending on where you live) but there are a lot of people there, so even though you are not very likely to know any particular inhabitant, it is nonetheless quite likely that you know at least one out of the whole population. This behavior is crucial to making the message-passing experiment work on this network.

Consider an individual in group A in Fig. 19.3. Let us suppose that, because of the effect above, this individual has at least one acquaintance at every "distance" in the tree, i.e., one acquaintance in every subtree of the hierarchy with which they share a common ancestor. That is, they know one of the individuals in group B, the one group with which they share ancestor 1, and they also know (say) someone in group C, one of the two with whom they share ancestor 2, and so on through groups D and E as shown. And suppose that a similar pattern of acquaintances holds for every individual in the network: everyone knows at least one person in every subtree with whom they share a common ancestor.

Now consider a greedy algorithm for message passing on this network. Suppose the message starts at a vertex in group A and, as before, the holder of the message at each step passes it to an acquaintance closer to the target than they are, distance now being measured in the sense of the hierarchical tree as described above.

Suppose the target vertex is in group X, which shares a common ancestor with A only at the highest and coarsest level marked 4 in the figure. That is, the target is in the opposite subtree of ancestor 4 from A. By hypothesis, the individual holding the message knows this and hence knows that in order to get the message closer to the target they must pass it to someone in that opposite subtree. Luckily, under the assumption above they always have such an acquaintance, in this case in group E. So they pass the message to their friend in group E. The friend now notes that the target X is in the subtree with whom they share the common ancestor marked 5 and hence knows that they must pass the message to a neighbor in that subtree to get it closer to the target. Again, by definition, they have at least one such neighbor, to whom they pass the message. And so the process proceeds. At each step we narrow down our search to a smaller subtree of the overall network, or equivalently we move to a lower level in the hierarchy, pivoting about a lower common ancestor. But the total number of levels in the hierarchy is $\log_2(n/g)$ and hence this is the maximum number of steps that the process will take to reach the target. In this model, therefore, the message always reaches its target in a logarithmic number of steps.

It's not, however, very realistic to assume that each individual in the network knows at least one person at each distance. Watts *et al.* considered a more realistic probabilistic model in which there is a probability $p_m$ of two individuals knowing one another when their lowest common ancestor is at level $m$ in the tree. The level is defined to be $m = 0$ for groups that are immediately adjacent, as A and B are in Fig. 19.3, and to increase by one for each higher level up to a maximum of $m = \log_2(n/g) - 1$ at the top of the tree.

Watts *et al.* considered the particular choice

$$p_m = C\,2^{-\beta m},$$

**(19.9)**

where $C$ and $\beta$ are constants.[273] So long as $\beta$ is positive this choice gives, as desired, a lower probability of acquaintance with more distant individuals, the exact rate of variation being controlled by the value of $\beta$. The parameter $C$ controls the overall number of acquaintances that each individual has.

The number of vertices with which any given vertex shares its ancestor at level $m$ is just $2^m g$ and hence the expected number of such vertices that it will be connected to is

$$2^m g p_m = C g 2^{(1-\beta)m}$$

**(19.10)**

with the choice above for $p_m$. Summing over all levels the total expected number of acquaintances an individual has, their average degree in the network, is

$$\langle k \rangle = C g \sum_{m=0}^{\log_2(n/g)-1} 2^{(1-\beta)m} = C g \frac{2^{(1-\beta)\log_2(n/g)} - 1}{2^{1-\beta} - 1} = C g \frac{(n/g)^{1-\beta} - 1}{2^{1-\beta} - 1}.$$

**(19.11)**

Thus the constant $C$ is given by

$$C = \frac{\langle k \rangle}{g} \frac{2^{1-\beta} - 1}{(n/g)^{1-\beta} - 1}.$$

**(19.12)**

In the limit of large $n$ this simplifies to

$$C = \begin{cases} (\langle k \rangle / g)(2^{1-\beta} - 1)(n/g)^{1-\beta} & \text{for } \beta < 1, \\ (\langle k \rangle / g) / \log_2(n/g) & \text{for } \beta = 1, \\ (\langle k \rangle / g)(1 - 2^{1-\beta}) & \text{for } \beta > 1. \end{cases}$$

**(19.13)**

Now if a particular vertex receives a message and wants to pass it to a member of the opposite

subtree at level $m$, it can do so provided it has a suitable acquaintance. If (19.10) is small, however, then most likely it will not, in which case the best it can do is to pass the message to someone else in the subtree it is already in, who can then repeat the process. The expected number of times this will happen before one person does have a neighbor in the opposite subtree is given by the reciprocal of (19.10), which is $2^{(\beta-1)m}/Cg$. Then, summing this over all levels, the total expected number of steps to reach the target is

$$\ell = \frac{1}{Cg} \sum_{m=0}^{\log_2(n/g)-1} 2^{(\beta-1)m} = \frac{1}{Cg} \frac{2^{(\beta-1)\log_2(n/g)} - 1}{2^{\beta-1} - 1}$$
$$= \frac{1}{Cg} \frac{(n/g)^{\beta-1} - 1}{2^{\beta-1} - 1}.$$

**(19.14)**

It is also possible that the vertex holding the message will not have a neighbor either in the opposite subtree or in its own subtree. If this happens then the vertex has only neighbors further from the target than it is and none nearer. In this case the Milgram experiment fails—recall that participants were asked to pass the message to someone closer to the target. This, however, is not necessarily unrealistic. As Watts *et al.* pointed out, this presumably does happen in the real experiment sometimes, and moreover it is well documented that many messages, a majority in fact, get lost and never reach their target. For messages that do get through, however, Eq. (19.14) gives an estimate of the number of steps they take to arrive.

Equation (19.14) is rather similar to the corresponding expression for the model of Kleinberg, Eq. (19.7), which is not a coincidence since the mechanisms by which the message-passing process proceeds are similar in the two cases. Taking the limit of large $n$ and making use of Eq. (19.13), we find that

$$\ell = \begin{cases} D(n/g)^{1-\beta} & \text{for } \beta < 1, \\ E\log^2(n/g) & \text{for } \beta = 1, \\ F(n/g)^{\beta-1} & \text{for } \beta > 1, \end{cases}$$

**(19.15)**

where $D$, $E$, and $f$ are constants.

These results have the same functional form as those of Eq. (19.8) for Kleinberg's model and tell us that it is indeed possible for Milgram's experiment to succeed in networks of this type, but only for the special parameter value $\beta = 1$. For all other values, the number of steps $\ell$ taken to reach the target increases as a power of $n$.

Thus the model of Watts *et al.* confirms Kleinberg's results in the context of a more realistic network. The results are, however, somewhat mysterious in a way. The idea that the network must be tuned to a special point in order for the Milgram experiment to succeed is surprising. The Milgram experiment does appear to succeed when conducted on real-world social networks, but on the face of it there is no clear reason why real-world networks should fall at this special point. Is it really true that if the world happened to be a little different from the way it is, Milgram's experiment would fail? This is a point that is not yet fully understood. It is possible that our model misses some important feature of the network structure that makes message passing more robust in

the real world and less dependent on the precise tuning of the network, or that people are using a different scheme for passing messages that works substantially better than our greedy algorithm. On the other hand, it is also possible that our model is basically correct but that the world is in fact only rather loosely tuned to the special point $\beta = 1$ at which message passing succeeds in finding short paths. For values of $\beta$ close to 1 the power of $n$ in Eq. (19.14) is small and hence $\ell$ still grows quite slowly. Indeed it is in general difficult to distinguish experimentally between low powers and logarithms, so any value of $\beta$ in the rough vicinity of $\beta = 1$ could result in good apparent performance in the message passing experiment.

## PROBLEMS

**19.1** Suppose that we use a web crawler to crawl a small portion of the Web, starting from a randomly chosen web page somewhere in the large in-component. Let us model the crawl as a breadth-first search starting from the given vertex and proceeding for $r$ "waves" of search, i.e., until it reaches vertices that are $r$ steps away from the start. Let $S_i$ be the size of the large in-component.

a. What is the probability that a given web page has been crawled at the "zeroth" wave of the algorithm, i.e., when only the one starting page has been crawled?
b. Argue that the probability $p_i$ that a page is first reached by the crawl on the rth wave is given approximately by $\mathbf{p}(r) = \mathbf{A}\mathbf{p}(r-1)$, where $\mathbf{p} = (p_1, p_2, ...)$. Why is this relation only approximate in general?
c. Hence argue that the probability of a page being found in a small crawl is roughly proportional to the eigenvector centrality of the page. Recall that the eigenvector centrality is zero for vertices in the in-component that don't also belong to the strongly connected component (see Section 7.2). Explain why this makes sense in the present context.

**19.2** Suppose that a search is performed on a peer-to-peer network using the following algorithm. Each vertex on the network maintains a record of the items held by each of its neighbors. The vertex originating a search queries one of its neighbors, chosen uniformly at random, for a desired item and the neighbor responds either that it or one of its neighbors has the item, in which case the search ends, or that they do not. In the latter case, the neighboring vertex then passes the query on to one of *its* neighbors, chosen at random, and the process repeats until the item is found. Effectively, therefore, the search query makes a random walk on the network.

a. Argue that, in the limit of a large number of steps, the probability that the query encounters a vertex $i$ on any particular step is $k_i/2m$, where $k_i$ is the degree as usual and $m$ is the total number of edges in the network.
b. Upon arriving at a vertex of degree $k$, the search learns (at most) about the items held by all of that vertex's $k$ neighbors except for the one the query is coming from, for a total of $k - 1$ vertices. Show that on average at each step the search learns about the contents of approximately $\langle k^2\rangle/\langle k\rangle - 1$ vertices and hence that, for a target item that can be found at a fraction $c$ of the vertices in the network, the expected number of copies of the item found on a given step is $c(\langle k^2\rangle/\langle k\rangle - 1)$.
c. Argue that the probability of not finding the target item on any particular step is approximately $q = \exp[c(1 - \langle k^2\rangle/\langle k\rangle)]$ and that average number of steps it takes to find a copy of the item is $1/(1 - q)$.
d. On a network with a power-law degree distribution with exponent less than 3, so that $\langle k^2\rangle \to \infty$, this last result implies that in the limit of large network size the search should end after only one step. Is this really true? If not, explain why not.

Although the random walk is not a realistic model for actual network search it is nonetheless useful: presumably more intelligent search strategies will find results quicker than a mindless

random walk and hence the random walk provides an upper bound on the length of search needed to find an item. In particular, if the random walk works well, as in the example above, then it suggests that more intelligent forms of search will also work well.

**19.3** The network navigation model of Kleinberg described in Section 19.3.1 is a one-dimensional version of what was, originally, a two-dimensional model. In Kleinberg's original version, the model was built on a two-dimensional square lattice with vertices connected by shortcuts with probability proportional to $r^{-\alpha}$ where $r$ is the "Manhattan" distance between the vertices, i.e., the geodesic network distance in terms of number of edges traversed (rather than the Euclidean distance). Following the outline of Section 19.3.1, sketch an argument to show for this variant of the model that it is possible to find a target vertex in $O(\log^2 n)$ steps, but only if $\alpha = 2$.

**19.4** Show that the ability to find short paths (of order $\log^2 n$) in the hierarchical model of Section 19.3.2 coincides with the state of the network in which a vertex has equal numbers of neighbors on average at each possible distance, where "distance" is defined by the lowest common ancestor two vertices share.

## REFERENCES


[1] Abello, J., Buchsbaum, A., and Westbrook, J., A functional approach to external graph algorithms, in *Proceedings of the 6th European Symposium on Algorithms*, Springer, Berlin (1998).

[2] Abramowitz, M. and Stegun, I. A., eds., *Handbook of Mathematical Functions*, Dover Publishing, New York (1974).

[3] Achlioptas, D., Clauset, A., Kempe, D., and Moore, C., On the bias of traceroute sampling: or, Power-law degree distributions in regular graphs, *J. ACM* **56**(4), 21 (2009).

[4] Adamic, L. A. and Glance, N., The political blogosphere and the 2004 US election, in *Proceedings of the WWW-2005 Workshop on the Weblogging Ecosystem* (2005).

[5] Adamic, L. A. and Huberman, B. A., The nature of markets in the World Wide Web, *Q. J. Electronic Commerce* **1**, 512 (2000).

[6] Adamic, L. A., Lukose, R. M., Puniyani, A. R., and Huberman, B. A., Search in power-law networks, *Phys. Rev. E* **64**, 046135 (2001).

[7] Adleman, L. M., Molecular computation of solutions to combinatorial problems, *Science* **266**, 1021- 1024 (1994).

[8] Ahuja, R. K., Magnanti, T. L., and Orlin, J. B., *Network Flows: Theory, Algorithms, and Applications*, Prentice Hall, Upper Saddle River, NJ (1993).

[9] Aiello, W., Chung, F., and Lu, L., A random graph model for massive graphs, in *Proceedings of the 32nd Annual ACM Symposium on Theory of Computing*, pp. 171-180, Association of Computing Machinery, New York (2000).

[10] Aiello, W., Chung, F., and Lu, L., Random evolution of massive graphs, in J. Abello, P. M. Pardalos, and M. G. C. Resende, eds., *Handbook of Massive Data Sets*, pp. 97-122, Kluwer, Dordrecht (2002).

[11] Albert, R. and Barabási, A.-L., Topology of evolving networks: Local events and universality, *Phys. Rev. Lett.* **85**, 5234-5237 (2000).

[12] Albert, R. and Barabási, A.-L., Statistical mechanics of complex networks, *Rev. Mod. Phys.* **74**, 47-97 (2002).

[13] Albert, R., Jeong, H., and Barabási, A.-L., Diameter of the world-wide web, *Nature* **401**, 130-131 (1999).

[14] Albert, R., Jeong, H., and Barabási, A.-L., Attack and error tolerance of complex networks, *Nature* **406**, 378-382 (2000).

[15] Aldous, D., Spatial transportation networks with transfer costs: Asymptotic optimality of hub and spoke models, *Mathematical Proceedings of the Cambridge Philosophical Society* **145**, 471-487 (2008).

[16] Amaral, L. A. N., Scala, A., Barthélémy, M., and Stanley, H. E., Classes of small-world networks, *Proc. Natl. Acad. Sci. USA* **97**, 11149-11152 (2000).